Pipeline Run Order (From Scratch)

This guide gives a **foolproof, start-to-finish** run path for the latest pipeline, with the updated UIC/prestige logic and FY backfill rules.

---

0) One-time prerequisites

- Verify the WDARFF 2015–2026 UIC tables are present in `./hierarchy_data/FMS_Web/WDARFF_15-26/`.
- Decide if you want **backfill** for pre-2015 years (see Section 4).

---

1) Build the UIC hierarchy (503)

Run: `503_hierarchies.ipynb`

Why: This is the **only place** the WDARFF 2015–2026 tables are read and converted to hierarchy lookup files.

Key outputs created by 503: - `df_uic_hierarchy` (UIC hierarchy table) - `df_uic_div_lookup` (FY-aware UIC → top_uic/div_name/class) - `prestige_uics_by_fy` (JSON in `./running_vars/`)

If 503 is not re-run, all downstream steps will use **old hierarchy data**.

---

2) Build the base snapshots (502)

Run: `502_working.ipynb`

Why: Produces the cleaned base snapshots and creates `prestige_unit` / `prestige_sum`.

Key outputs: - `df_502_base.feather` (used by 520)

Important: 502 *loads* `df_uic_hierarchy` and uses it to assign `prestige_unit`.

---

3) Main pipeline (520)

Run: `520_pipeline_cox_working.ipynb`

Minimal flow: 1. **Cell 0** (imports + config) 2. **Cell 7** (time-varying variables) 3. **Cell 9–12** (analysis, plots, models)

Key integration points: - Cell 7 calls `add_cum_oer_metrics_mod_working.py` and computes prestige-based pool metrics. - Cell 7 can also add `div_name` and division metrics via `py_503_hierarchies.apply_division_name()` when enabled in `pipeline_config.py`.

Flow chart: z-scores, plots, and models

```
pipeline_config.py
  ├─ STANDARDIZE_CONFIG
  |    └─ Cell 10/10.5 creates z_ columns for:
  |         - STANDARDIZE_CONFIG['cols'] (if provided), else
  |         - model_static_cols + model_time_varying_cols
  |
  ├─ COLUMN_CONFIG (time_varying_cols)
  |    └─ Cell 11 builds df_analysis from:
  |         basic_demographic_cols + static_cols + time_varying_cols
  |
  └─ PLOT_CONFIG['plots']
       └─ Cell 11 uses these for KM/CR plots (descriptive)

Cell 12 (models)
  ├─ If model_*_cols are set → use them
  └─ Else → derive from PLOT_CONFIG['plots'] (variable + group_by)
```

Key takeaway: - **Standardization does not automatically add z_ columns to Cell 11.** - **Cell 11 only sees what is in COLUMN_CONFIG['time_varying_cols']**. - **Cell 12 uses model_*_cols if provided; otherwise plot-derived defaults.**

---

4) FY backfill rules (2002–2014)

Prestige backfill (union of 2015–2026 UICs)

Configured in `pipeline_config.py`: - `PRESTIGE_CONFIG['backfill_early_fy'] = True` - `PRESTIGE_CONFIG['backfill_use_union'] = True` (default) - `PRESTIGE_CONFIG['backfill_start_fy'] = 2002`

This **only affects prestige_unit** assignment.

Division backfill (FY 2015 only)

Configured in `pipeline_config.py`: - `DIVISION_CONFIG['use_fy_lookup'] = True` - `DIVISION_CONFIG['backfill_early_fy'] = True` - `DIVISION_CONFIG['backfill_fy'] = 2015`

This applies **only to division names/metrics**, not prestige.

---

5) Optional: Auto-run 503 from 520

If you want 520 to build the prestige list automatically each run:

```
RUN_503 = True
```

This calls `py_503_hierarchies.get_prestige_uics_by_fy(...)` and writes the JSON to `./running_vars/`.

---

6) Quick validation checks

After running the full pipeline:

```
# Prestige unit exists?
df_cox['prestige_unit'].value_counts(dropna=False).head()

# Prestige ratio exists?
df_cox['prestige_quarters_ratio'].isna().mean()

# Division name (if enabled)?
df_cox['div_name'].isna().mean()
```

If `prestige_quarters_ratio` is all NaN, verify: - `prestige_unit` exists - `include_prestige_pool_metrics` is True (CELL 6C)

---

7) Minimal “from-scratch” checklist

1. Run `503_hierarchies.ipynb`
2. Run `502_working.ipynb`
3. Run `520_pipeline_cox_working.ipynb` (Cells 0 → 12)